THE UNIVERSITY OF ALABAMA®

WHERE LEGENDS ARE MADE®

Jiacheng Li, Ph.D.

Red-Black Trees

CS 201: Data Structures and Algorithms

Jiacheng Li, Ph.D.

A Balanced Tree

- A *balanced tree* in computer science refers to a type of binary tree structure that maintains its height to be as low as possible while elements are added or removed, ensuring that operations like search, insert, and delete can be performed efficiently.
- The primary goal of a balanced tree is to maintain its height around $O(\log n)$, where n is the number of nodes.

Red-Black Trees

- A *Red-Black Tree* is a type of self-balancing binary search tree.
- It maintains *several additional properties* to ensure that the tree remains approximately balanced, which helps keep the operations such as insertion, deletion, and searching efficient, generally within $O(\log n)$ complexity, where n is the number of nodes in the tree.

Properties of red-black trees

A red-black tree is a binary search tree that satisfies the following **Red-black** properties:

1. Every node is either red or black.
2. The root is black.
3. Every leaf (NIL) is black.
4. If a node is red, then both its children are black.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.

Proposition: The height of a red-black tree with n internal nodes is $\leq 2 * \lg(n+1) = O(\lg n)$.

Sample Red-Black Tree

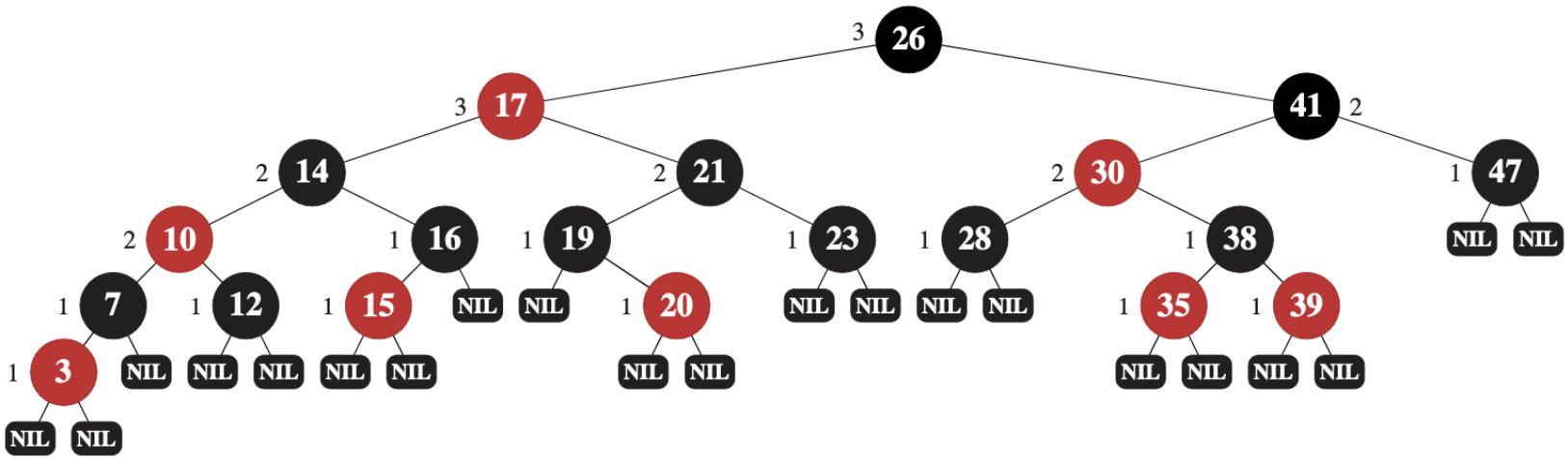


Figure 1: A Red-black tree with explicit NIL leaves.

1. Every node is either **red** or **black**.
2. The root is always **black**.
3. Every leaf (NIL node) is **black**.
4. If a node is **red**, both of its children must be **black**.
5. For each node, all paths from the node to its descendant leaves must have the same number of black nodes (called the black-height).

A Common Red-Black Tree

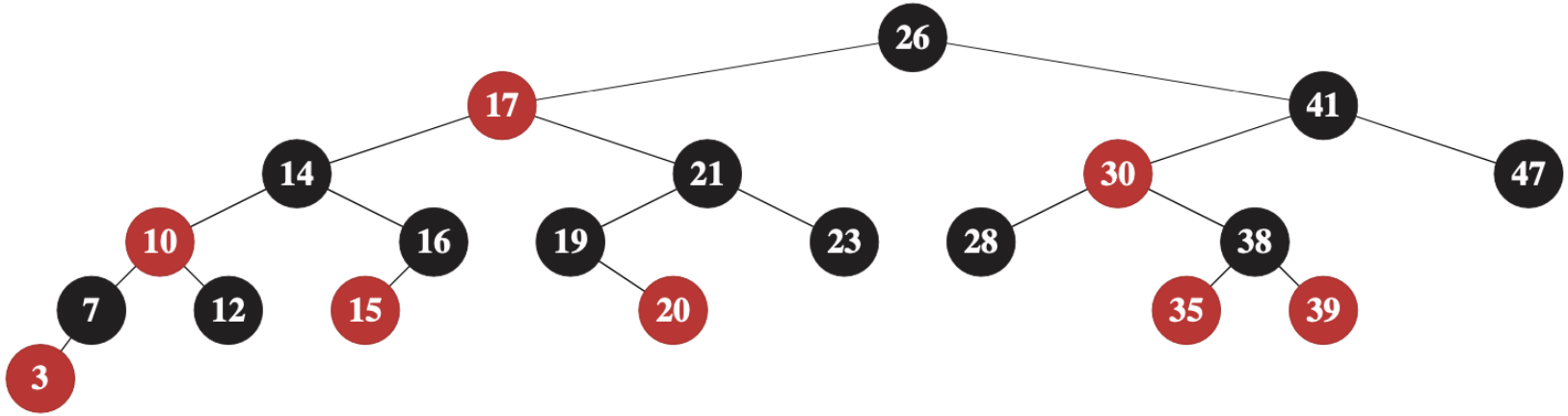


Figure: A red-black tree with implicit NIL leaves.

1. Every node is either **red** or **black**.
2. The root is always **black**.
3. Every leaf (NIL node) is **black**.
4. If a node is **red**, both of its children must be **black**.
5. For each node, all paths from the node to its descendant leaves must have the same number of black nodes (called the black-height).

Red-Black Trees

- The rotation function in a Red-Black Tree is essential for maintaining the tree's balance after insertions or deletions.
- In Red-Black Trees, the goal is to keep the tree's height approximately logarithmic, which ensures that search, insert, and delete operations can all be performed in $O(\lg n)$ time.
- Rotation functions:
  - A Left Rotation is applied when the tree becomes right-heavy (i.e., when a node has a heavy right subtree, causing the tree to become unbalanced).
  - A Right Rotation is applied when the tree becomes left-heavy (i.e., when a node has a heavy left subtree).

Rotations

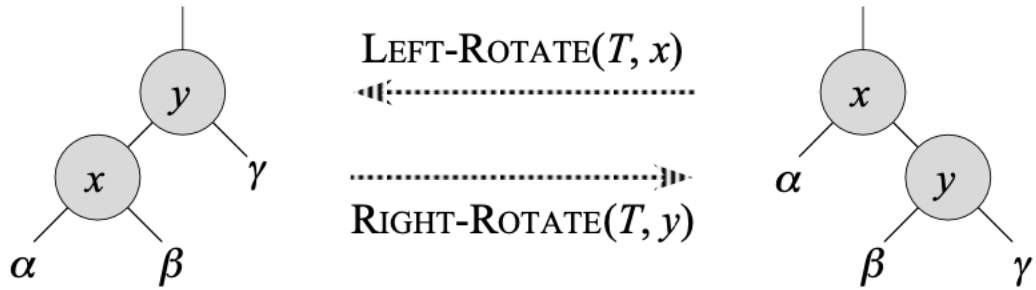


Figure: The rotation operations on a binary search tree.

Rotation preserves BST property

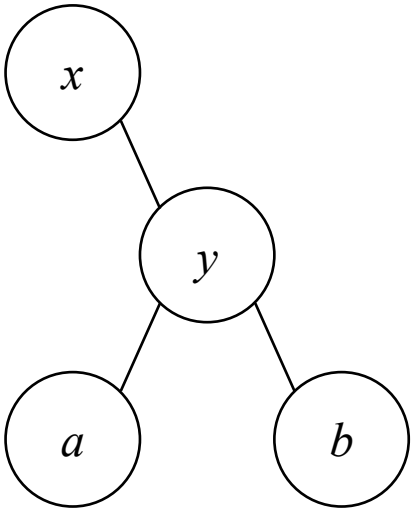
$$\alpha < x < \beta < y < \gamma$$

Rotations - Pseudocode

```
LEFT-ROTATE(T, x)
y = x.right                    // set y
x.right = y.left               // turn y's left subtree into x's right subtree
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p                      // link x's parent to y
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x                     // put x on y's left
x.p = y
```

Rotations

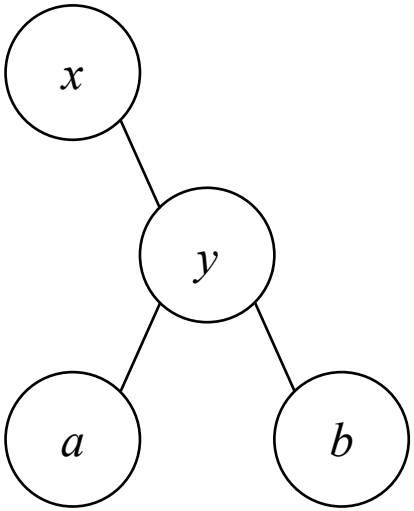
```
LEFT-ROTATE(T, x)
    y = x.right                 Set y
    x.right = y.left
    if y.left ≠ T.nil
        y.left.p = x
    y.p = x.p
    if x.p == T.nil
        T.root = y
    elseif x == x.p.left
        x.p.left = y
    else x.p.right = y
    y.left = x
    x.p = y
```



Rotations

turn y's left subtree into x's right subtree

```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x
x.p = y
```

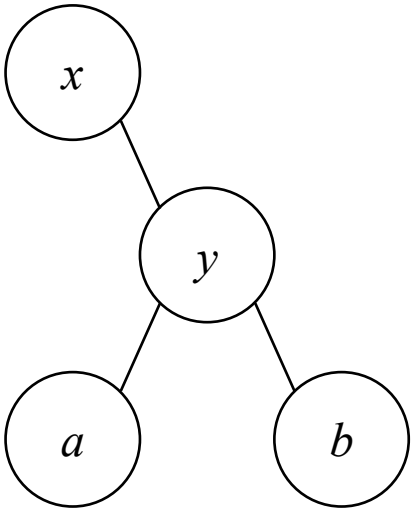


$x.right = y$.left;
$x.right =$ a;

Rotations

turn y's left subtree into x's right subtree

```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x
x.p = y
```



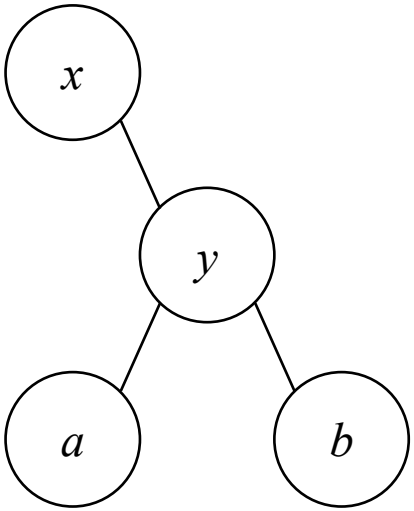
$x.right = y$.left;
$x.right =$ a;

$y.left.p = x$;
$a.p = x$;

Rotations

turn y's left subtree into x's right subtree

```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x
x.p = y
```



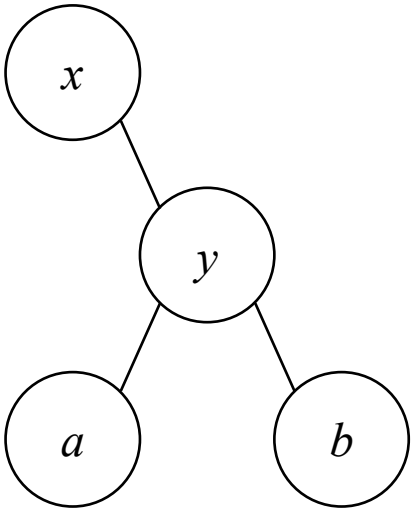
$x.right = y$.left;
$x.right$ = a;

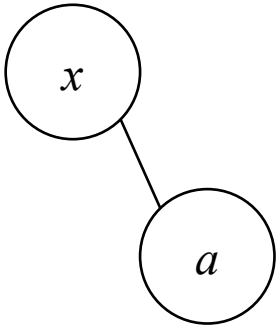
$y.left.p = x$;
$a.p = x$;

Rotations

turn y's left subtree into x's right subtree

```
LEFT-ROTATE(T, x)
    y = x.right
    x.right = y.left
    if y.left ≠ T.nil
        y.left.p = x
    y.p = x.p
    if x.p == T.nil
        T.root = y
    elseif x == x.p.left
        x.p.left = y
    else x.p.right = y
    y.left = x
    x.p = y
```



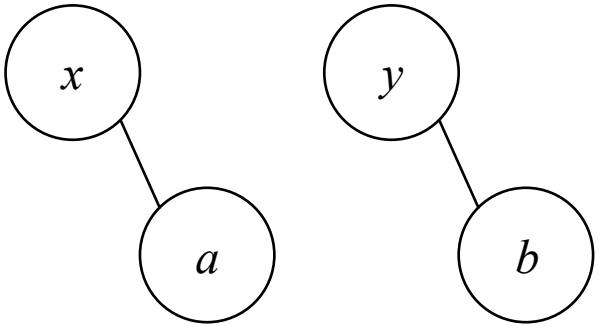


$x.right = y.\text{left};$
$x.right = \text{a};$

$y.left.p = x;$
$a.p = x;$

Rotations

```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x
x.p = y
```

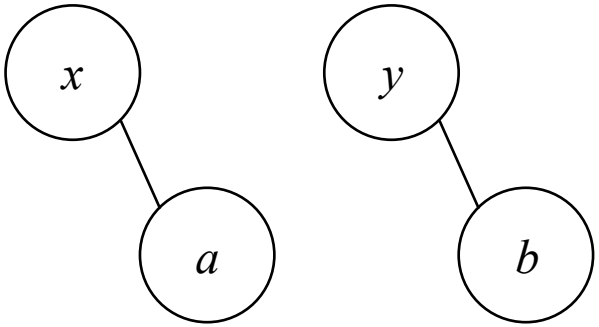


y is temporarily disconnected.

Rotations

```
LEFT-ROTATE(T, x)
    y = x.right
    x.right = y.left
    if y.left ≠ T.nil
        y.left.p = x
    y.p = x.p
    if x.p == T.nil
        T.root = y
    elseif x == x.p.left
        x.p.left = y
    else x.p.right = y
    y.left = x
    x.p = y
```

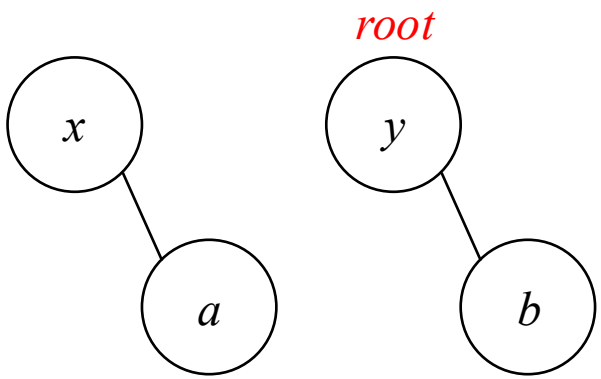
link x's parent to y.



y is temporarily disconnected.

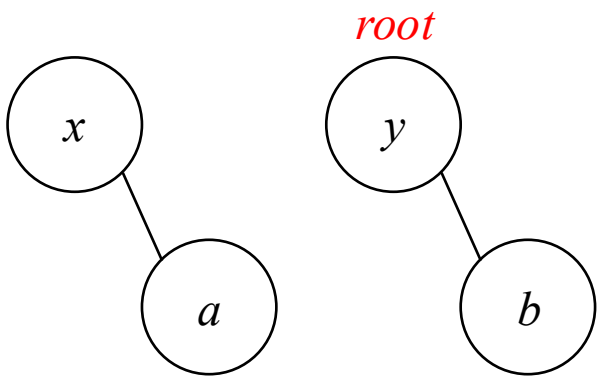
Rotations

```
LEFT-ROTATE(T, x)
    y = x.right
    x.right = y.left
    if y.left ≠ T.nil
        y.left.p = x
    y.p = x.p
    if x.p == T.nil
        T.root = y
    elseif x == x.p.left
        x.p.left = y
    else x.p.right = y
    y.left = x
    x.p = y
```



Rotations

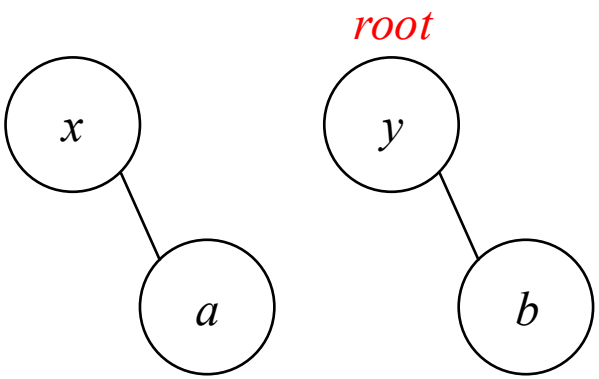
```
LEFT-ROTATE(T, x)
    y = x.right
    x.right = y.left
    if y.left ≠ T.nil
        y.left.p = x
    y.p = x.p
    if x.p == T.nil
        T.root = y
    elseif x == x.p.left
        x.p.left = y
    else x.p.right = y
    y.left = x
    x.p = y
```



Rotations

```
LEFT-ROTATE(T, x)
    y = x.right
    x.right = y.left
    if y.left ≠ T.nil
        y.left.p = x
    y.p = x.p
    if x.p == T.nil
        T.root = y
    elseif x == x.p.left
        x.p.left = y
    else x.p.right = y
    y.left = x
    x.p = y
```

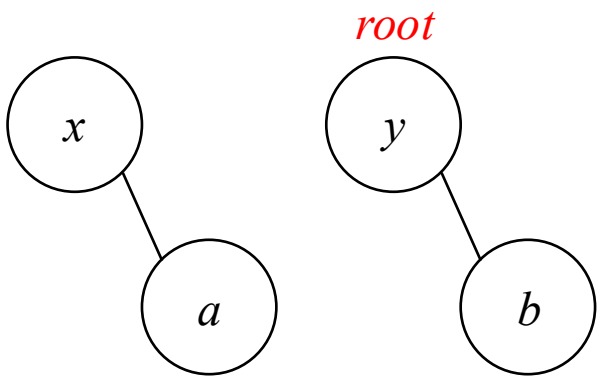
put x on y's left

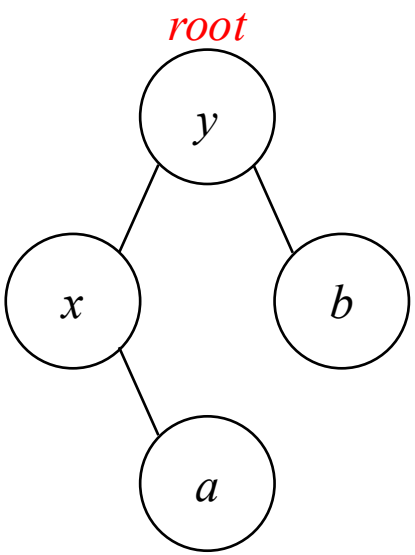


Rotations

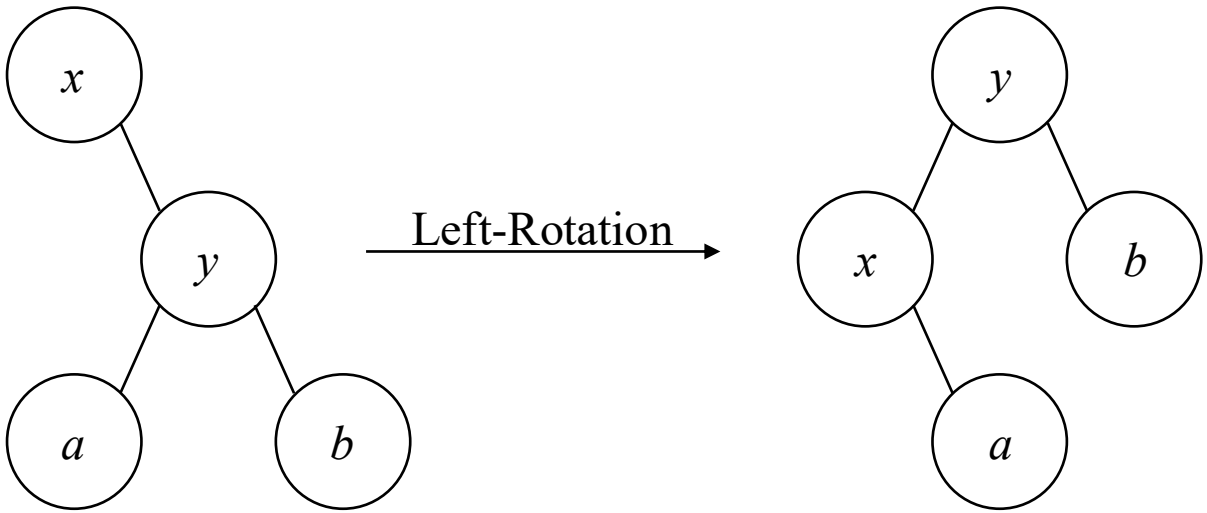
```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left
if y.left ≠ T.nil
    y.left.p = x
y.p = x.p
if x.p == T.nil
    T.root = y
elseif x == x.p.left
    x.p.left = y
else x.p.right = y
y.left = x
x.p = y
```

put x on y's left



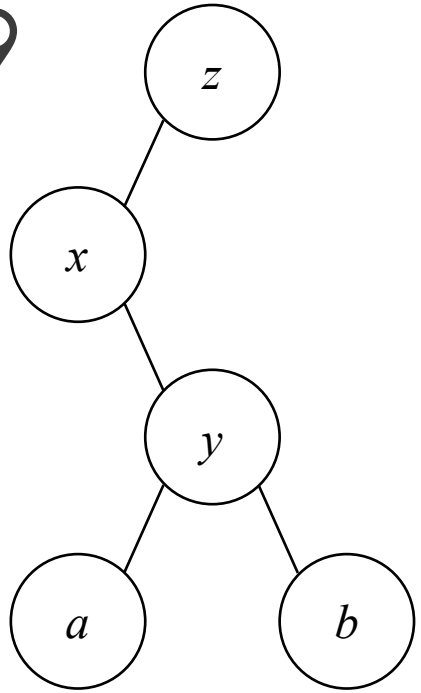


Rotations - finished



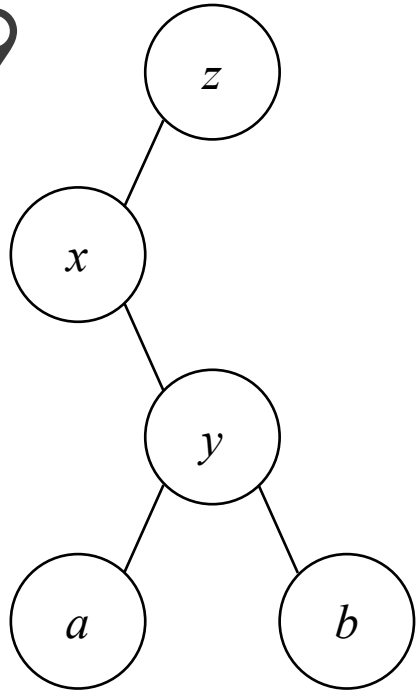
Rotations - lines 8&9

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```



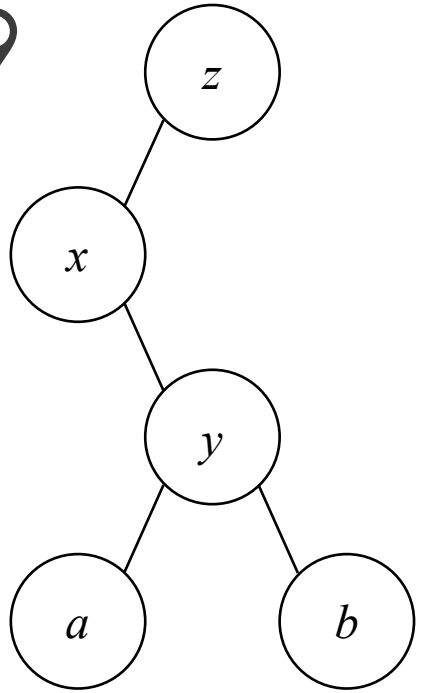
Rotations - lines 8&9

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```



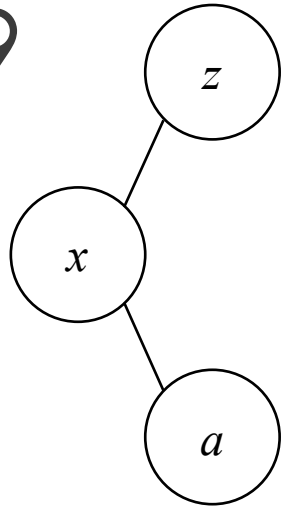
Rotations - lines 8&9

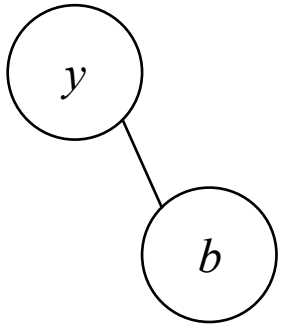
```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```



Rotations - lines 8&9

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```

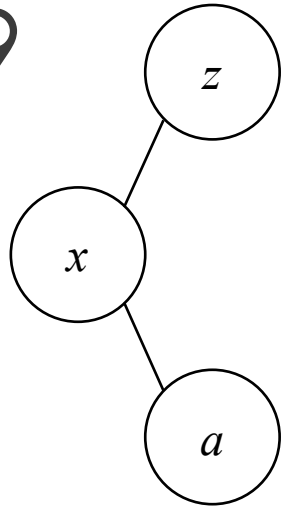


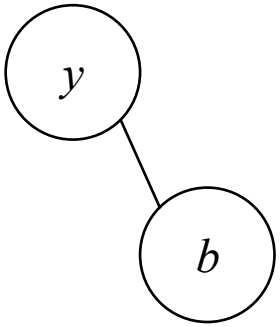


y is temporarily disconnected.

Rotations - lines 8&9

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```

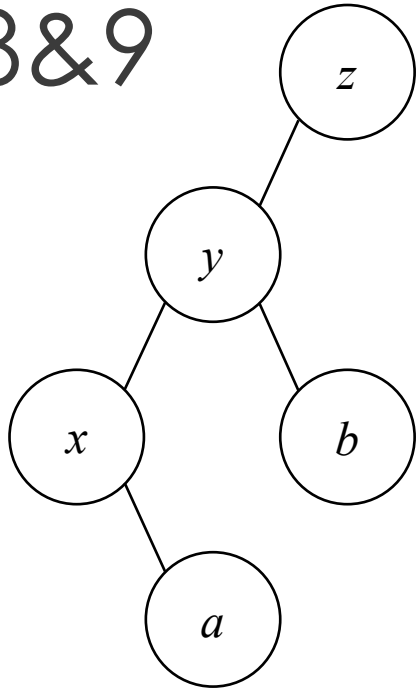




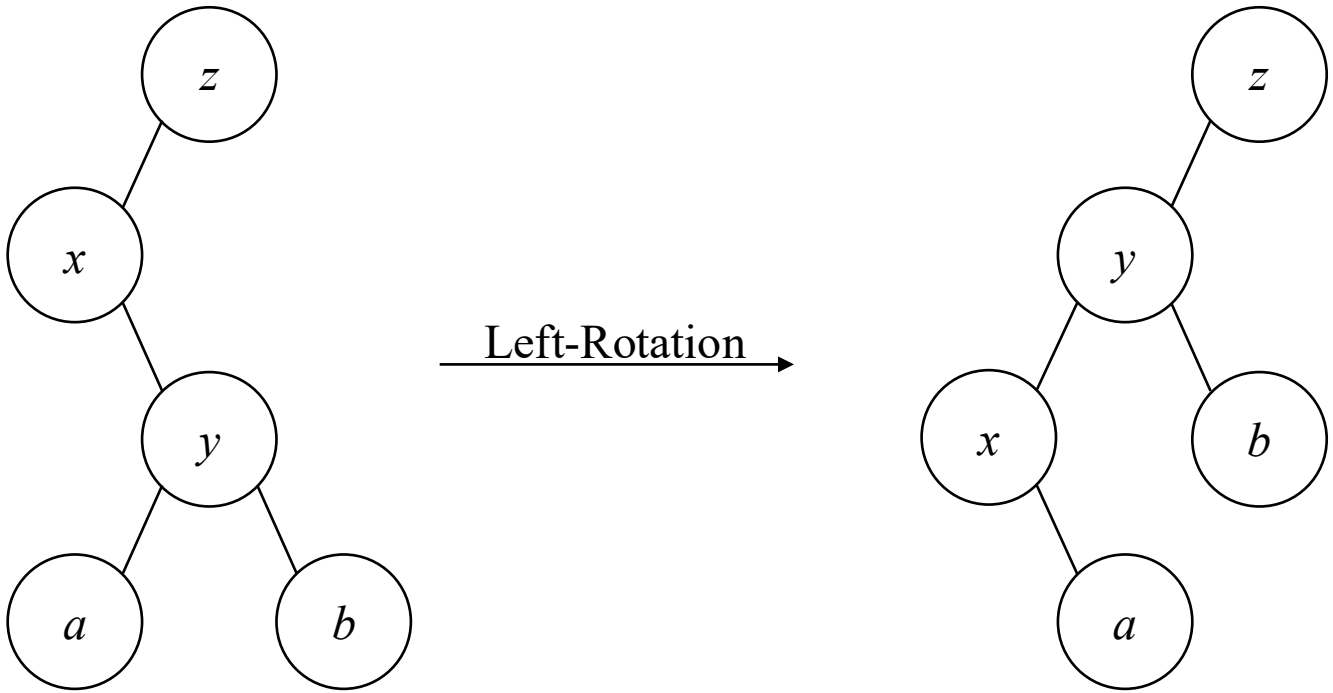
y is temporarily disconnected.

Rotations - lines 8&9

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```

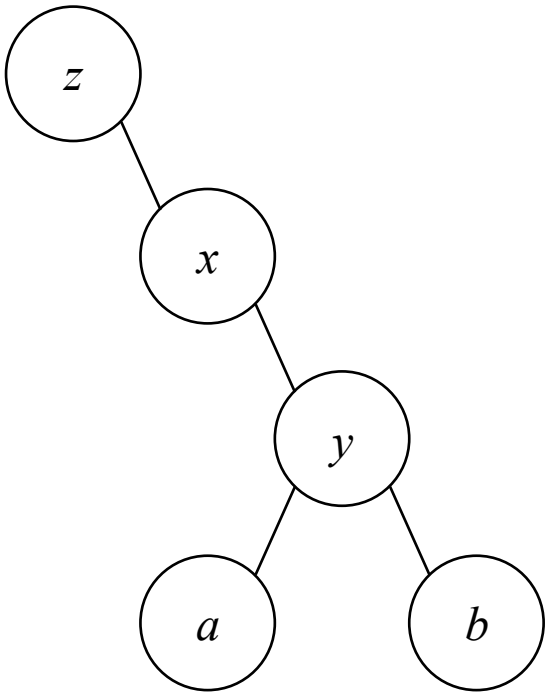


Rotations - finished

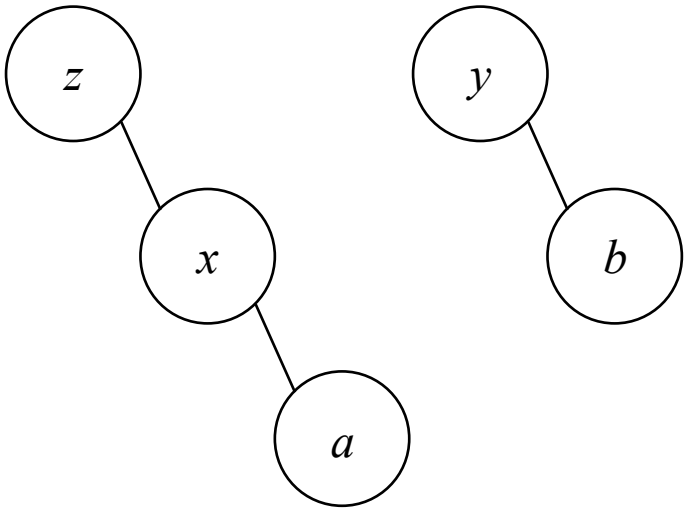


Rotations - line 10

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```

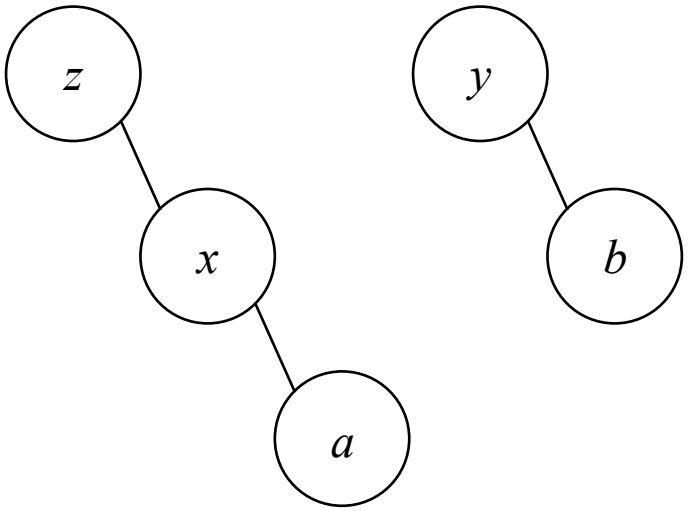


Rotations - line 10




```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```

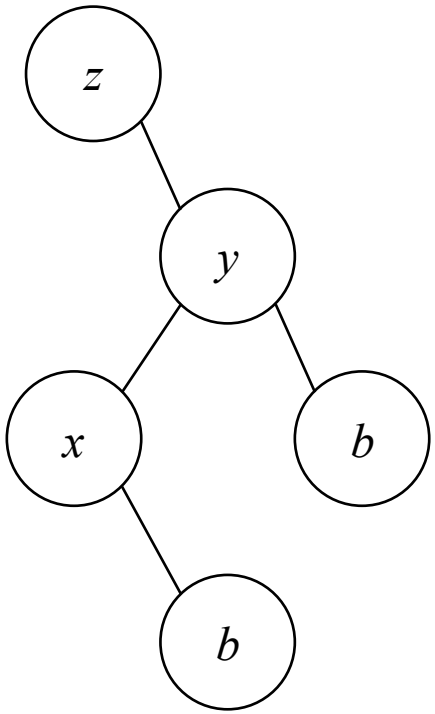
Rotations - line 10



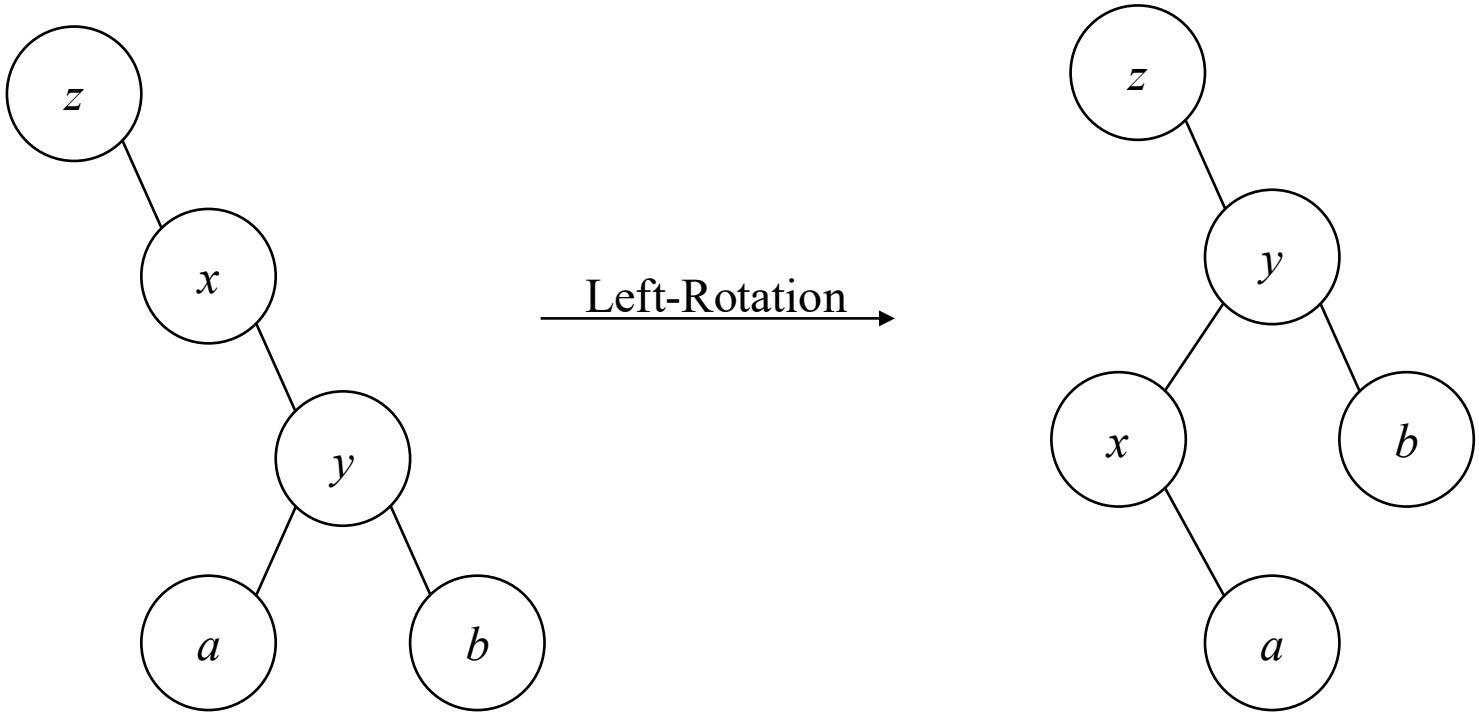

```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```

Rotations - line 10

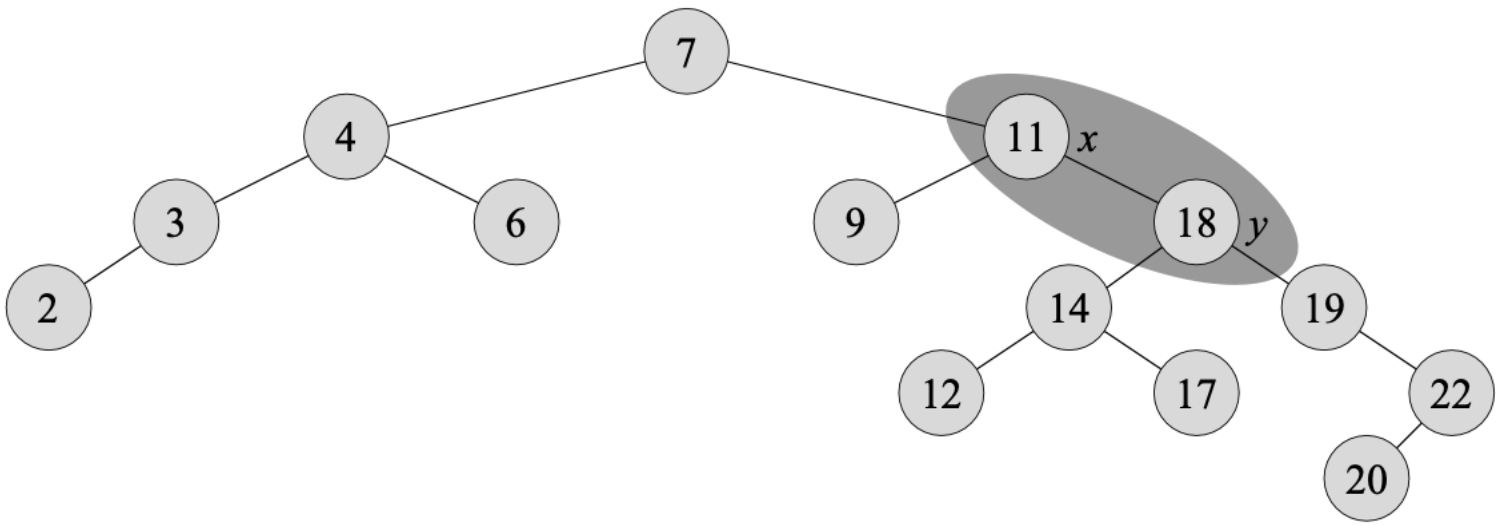
```
LEFT-ROTATE(T, x)
1   y = x.right
2   x.right = y.left
3   if y.left ≠ T.nil
4       y.left.p = x
5   y.p = x.p
6   if x.p == T.nil
7       T.root = y
8   elseif x == x.p.left
9       x.p.left = y
10  else x.p.right = y
11  y.left = x
12  x.p = y
```



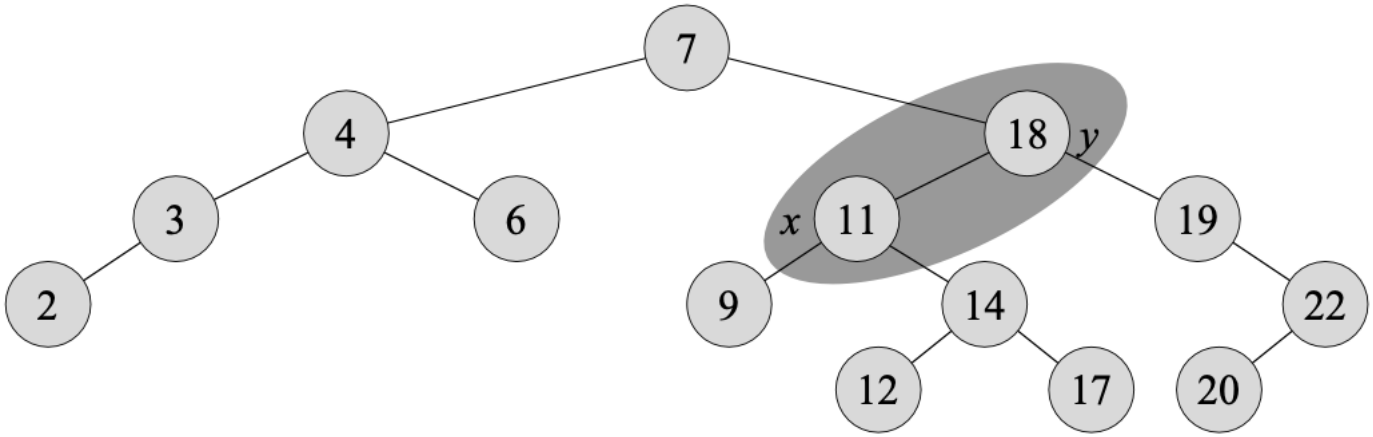
Rotations - finished



Rotations - Practice



Rotations - Practice



Rotations – Pseudocode (L)

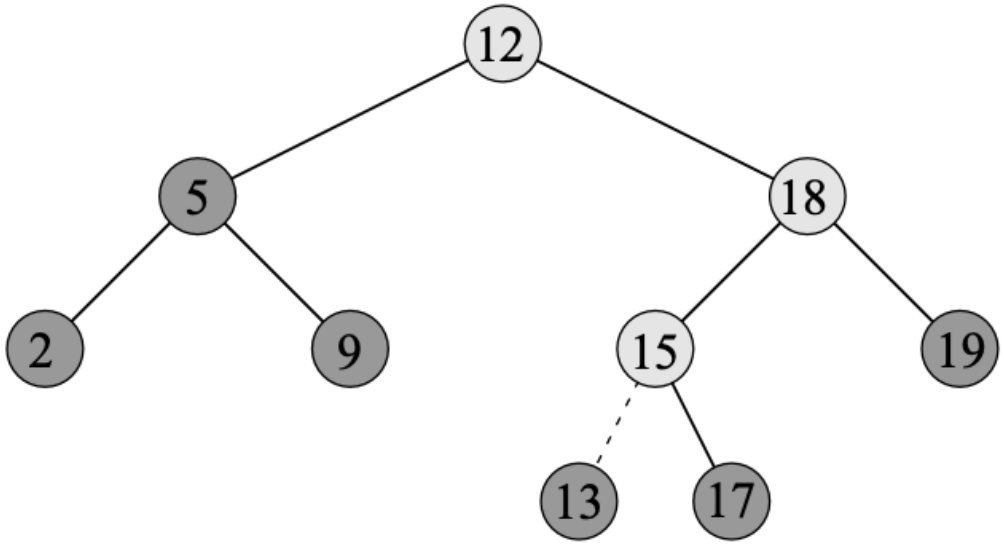
```
LEFT-ROTATE(T, x)
y = x.right
x.right = y.left
if y.left != T.nil
    y.left.p = x
y.p = x.p
if x.p == T.nil
    T.root = y
else if x == x.p.left
    x.p.left = y
else
    x.p.right = y
y.left = x
x.p = y
```

Rotations – Pseudocode (R)

```
RIGHT-ROTATE(T, y)
x = y.left
y.left = x.right
if x.right != T.nil
    x.right.p = y
x.p = y.p
if y.p == T.nil
    T.root = x
else if y == y.p.right
    y.p.right = x
else
    y.p.left = x
x.right = y
y.p = x
```

Insertion

```
TREE-INSERT(T, z)
y = NIL
x = T.root
while x ≠ NIL
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == NIL
    T.root = z          // tree T was empty
elseif z.key < y.key
    y.left = z
else y.right = z
```



Insertion

```
TREE-INSERT(T, z)
y = NIL
x = T.root
while x ≠ NIL
    y = x
    if z.key < x.key
        x = x.left
    else x = x.right
z.p = y
if y == NIL
    T.root = z          // tree T was empty
elseif z.key < y.key
    y.left = z
else y.right = z
```

```
RB-INSERT(T, z)
1  y = T.nil
2  x = T.root
3  while x ≠ T.nil
4      y = x
5      if z.key < x.key
6          x = x.left
7      else x = x.right
8  z.p = y
9  if y == T.nil
10     T.root = z
11 elseif z.key < y.key
12     y.left = z
13 else y.right = z
14 z.left = T.nil
15 z.right = T.nil
16 z.color = RED
17 RB-INSERT-FIXUP(T, z)
```

Because coloring z red may violate one of the red-black properties, we call RB-INSERT-FIXUP(T. z) in line 17 of RB-INSERT to restore the red-black properties.

Dr. Alexander Ororbia, PhD

Insertion

The **RB-INSERT-FIXUP** algorithm ensures that the tree remains balanced after the insertion of a new node z. The key properties of a Red-Black Tree are:

1. Every node is either **red** or **black**.
2. The root is always **black**.
3. Every leaf (NIL node) is **black**.
4. If a node is **red**, both of its children must be **black**.
5. For each node, all paths from the node to its descendant leaves must have the same number of black nodes (called the black-height).

After inserting a node, some of these properties might be violated. The algorithm works to restore the properties by adjusting the tree structure through **color changes** and **rotations**.

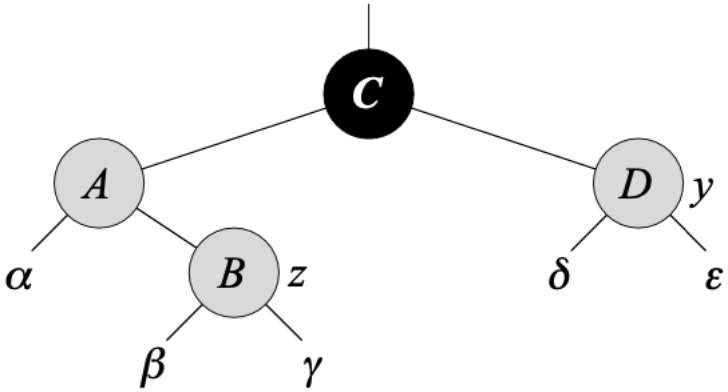
```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK                    // case 1
            y.color = BLACK                      // case 1
            z.p.p.color = RED                    // case 1
            z = z.p.p                            // case 1
        else if z == z.p.right
                z = z.p                          // case 2
                LEFT-ROTATE(T, z)                // case 2
            z.p.color = BLACK                    // case 3
            z.p.p.color = RED                    // case 3
            RIGHT-ROTATE(T, z.p.p)               // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Insertion

The **RB-INSERT-FIXUP** algorithm ensures that the tree remains balanced after the insertion of a new node z. The key properties of a Red-Black Tree are:

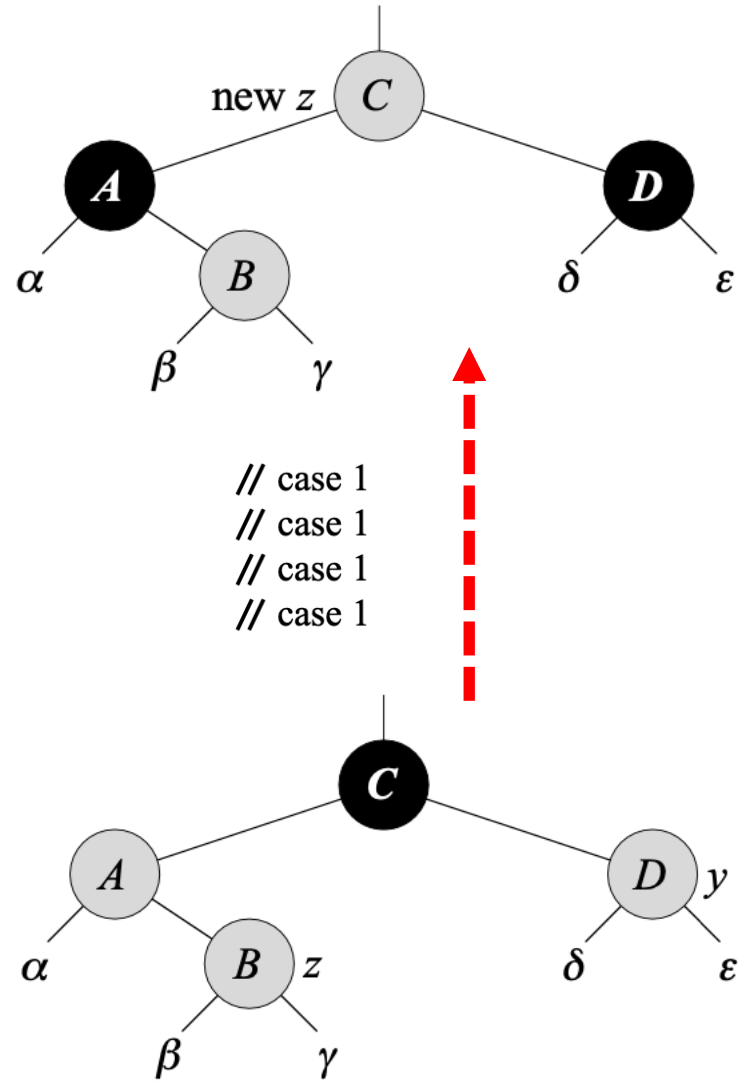
1. Every node is either **red** or **black**.
2. The root is always **black**.
3. Every leaf (NIL node) is **black**.
4. If a node is **red**, both of its children must be **black**.
5. For each node, all paths from the node to its descendant leaves must have the same number of black nodes (called the black-height).

After inserting a node, some of these properties might be violated. The algorithm works to restore the properties by adjusting the tree structure through **color changes** and **rotations**.



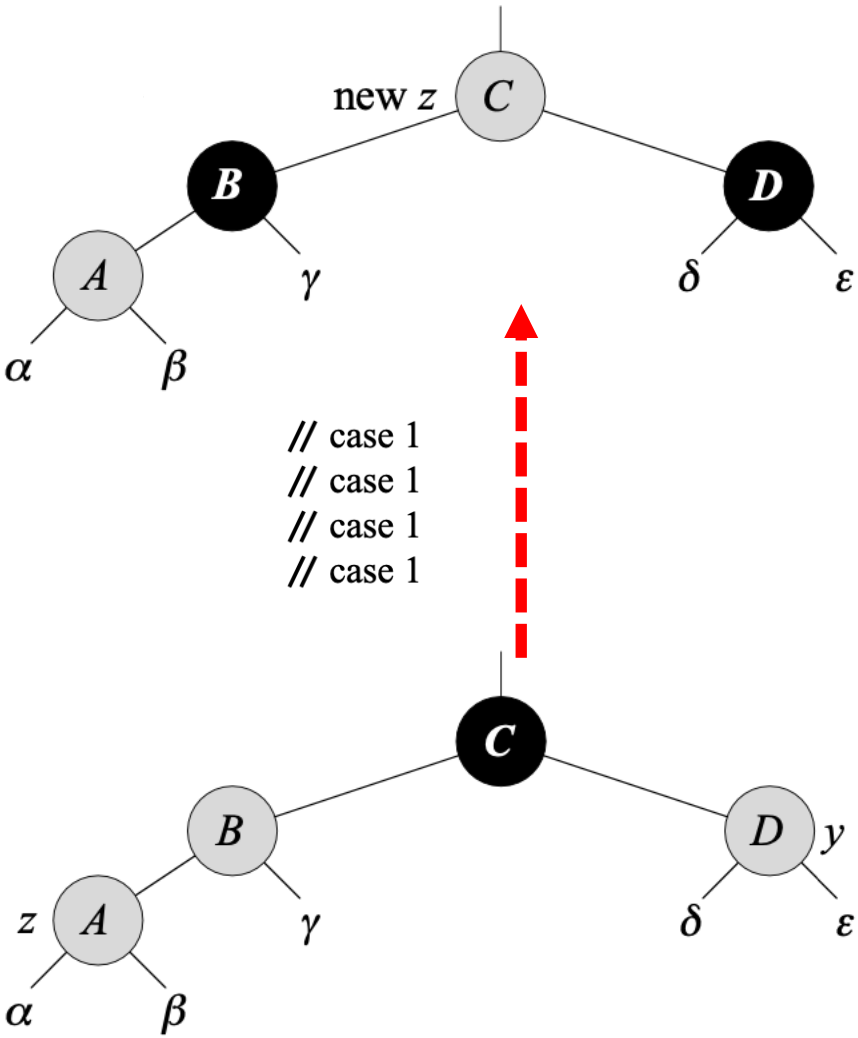
Insertion

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```



Insertion

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p
                LEFT-ROTATE(T, z)
            z.p.color = BLACK
            z.p.p.color = RED
            RIGHT-ROTATE(T, z.p.p)
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```



Insertion

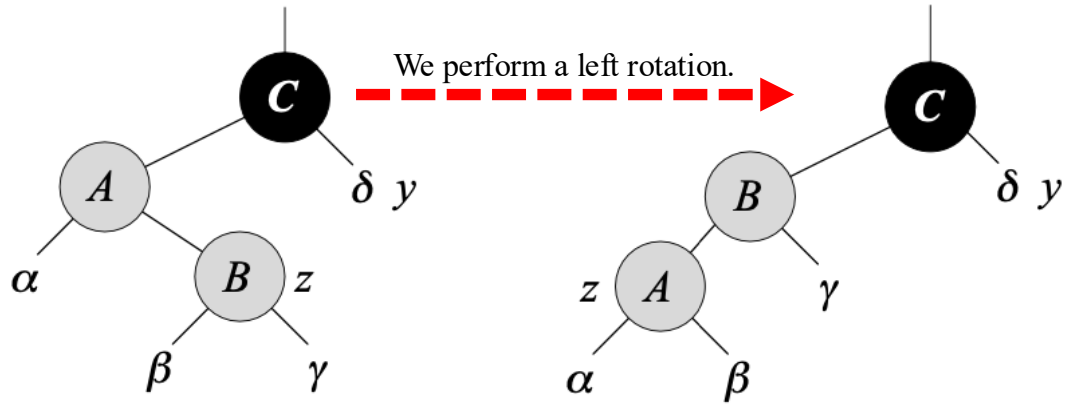
The **RB-INSERT-FIXUP** algorithm ensures that the tree remains balanced after the insertion of a new node z. The key properties of a Red-Black Tree are:

1. Every node is either **red** or **black**.
2. The root is always **black**.
3. Every leaf (NIL node) is **black**.
4. If a node is **red**, both of its children must be **black**.
5. For each node, all paths from the node to its descendant leaves must have the same number of black nodes (called the black-height).

After inserting a node, some of these properties might be violated. The algorithm works to restore the properties by adjusting the tree structure through **color changes** and **rotations**.

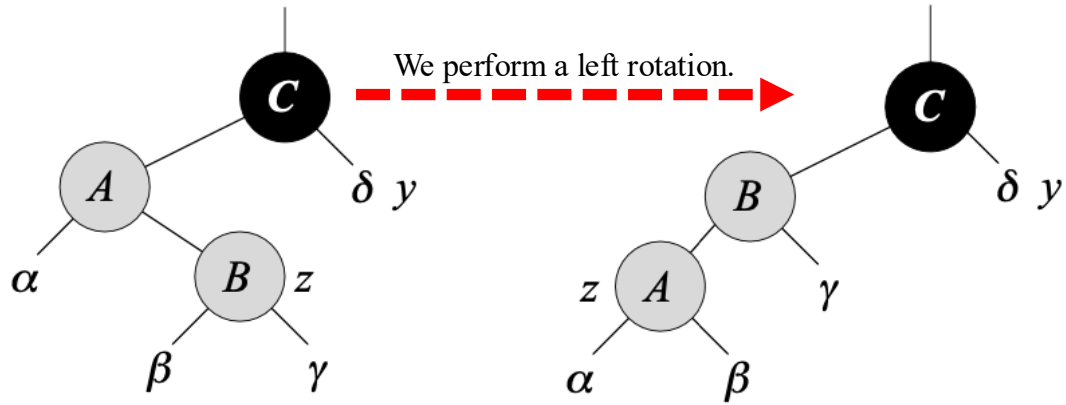
```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK                    // case 1
            y.color = BLACK                      // case 1
            z.p.p.color = RED                    // case 1
            z = z.p.p                            // case 1
        else if z == z.p.right
                z = z.p                          // case 2
                LEFT-ROTATE(T, z)                // case 2
            z.p.color = BLACK                    // case 3
            z.p.p.color = RED                    // case 3
            RIGHT-ROTATE(T, z.p.p)               // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Insertion



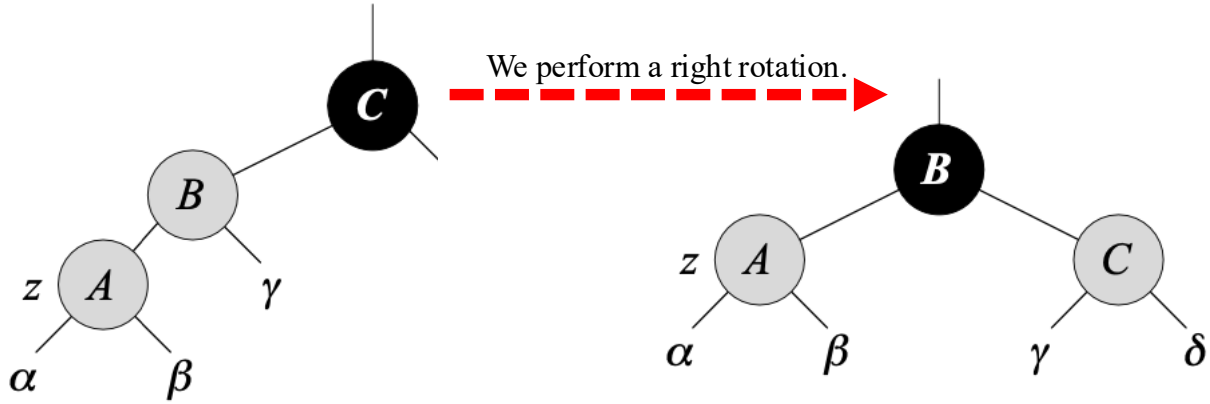

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p                                  // case 2
                LEFT-ROTATE(T, z)                        // case 2
            z.p.color = BLACK                            // case 3
            z.p.p.color = RED                            // case 3
            RIGHT-ROTATE(T, z.p.p)                       // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Insertion



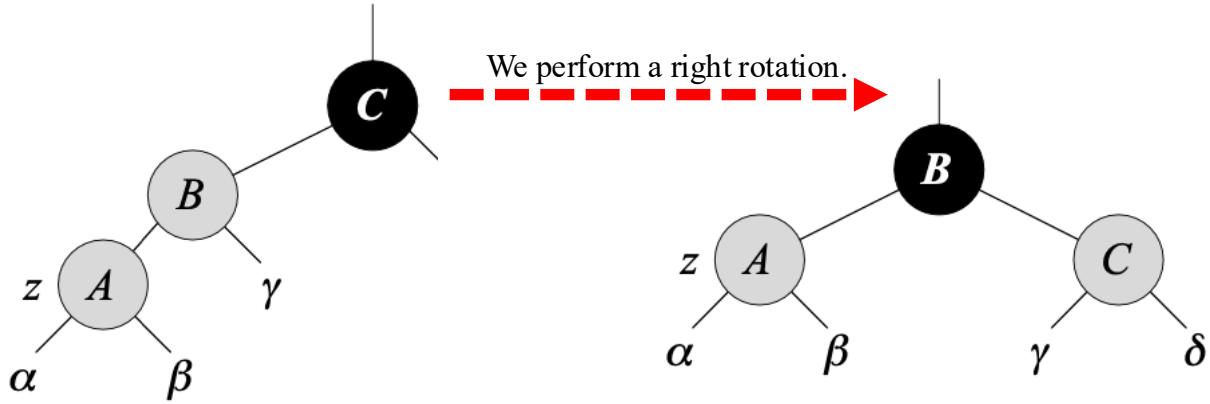

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p                                  // case 2
                LEFT-ROTATE(T, z)                        // case 2
            z.p.color = BLACK                            // case 3
            z.p.p.color = RED                            // case 3
            RIGHT-ROTATE(T, z.p.p)                       // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Insertion



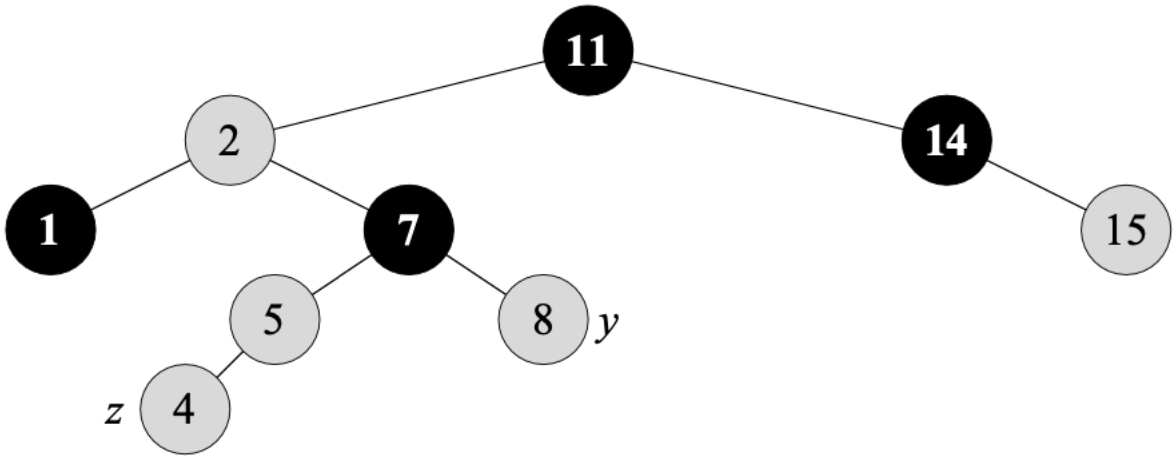

```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p                                  // case 2
                LEFT-ROTATE(T, z)                        // case 2
            z.p.color = BLACK                            // case 3
            z.p.p.color = RED                            // case 3
            RIGHT-ROTATE(T, z.p.p)                       // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

Insertion

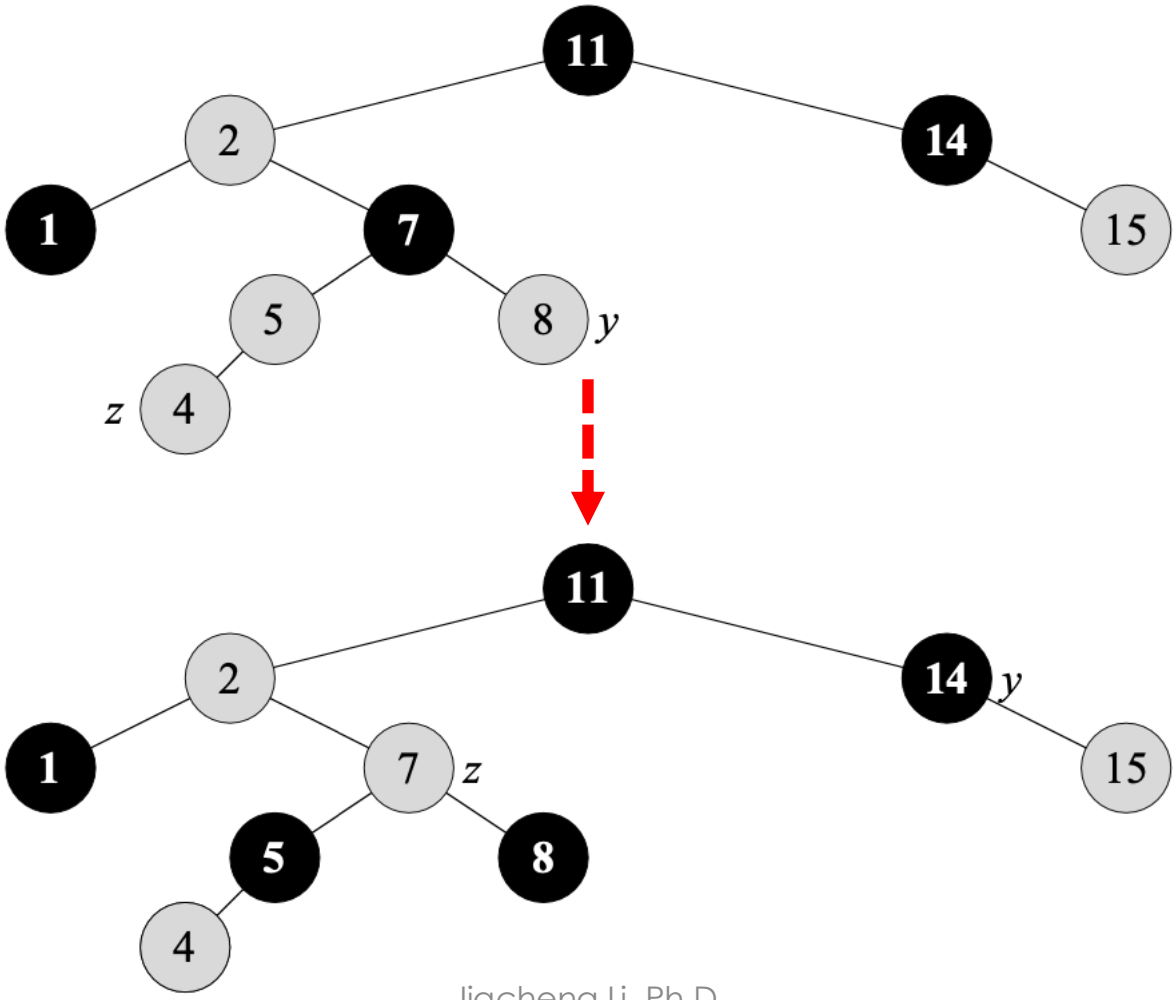



```
RB-INSERT-FIXUP(T, z)
while z.p.color == RED
    if z.p == z.p.p.left
        y = z.p.p.right
        if y.color == RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED
            z = z.p.p
        else if z == z.p.right
                z = z.p                                  // case 2
                LEFT-ROTATE(T, z)                        // case 2
            z.p.color = BLACK                            // case 3
            z.p.p.color = RED                            // case 3
            RIGHT-ROTATE(T, z.p.p)                       // case 3
    else (same as then clause
            with "right" and "left" exchanged)
T.root.color = BLACK
```

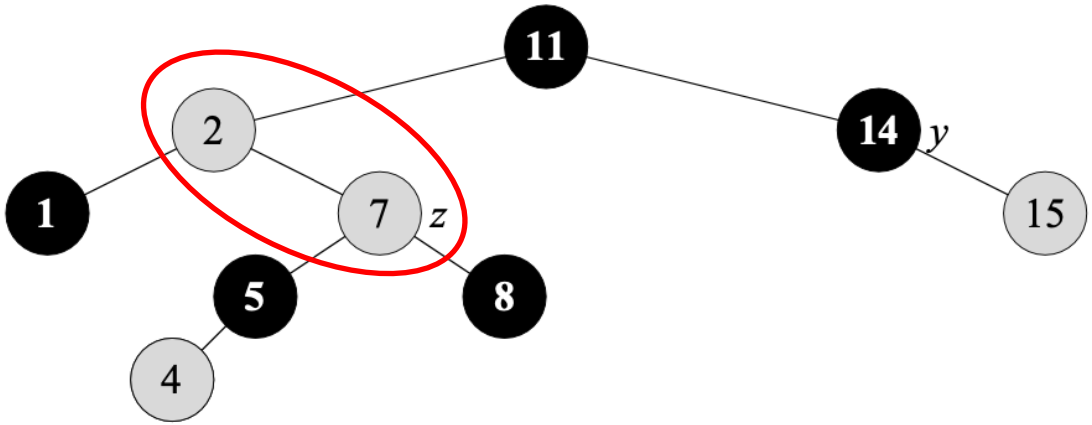
Insertion - Practice



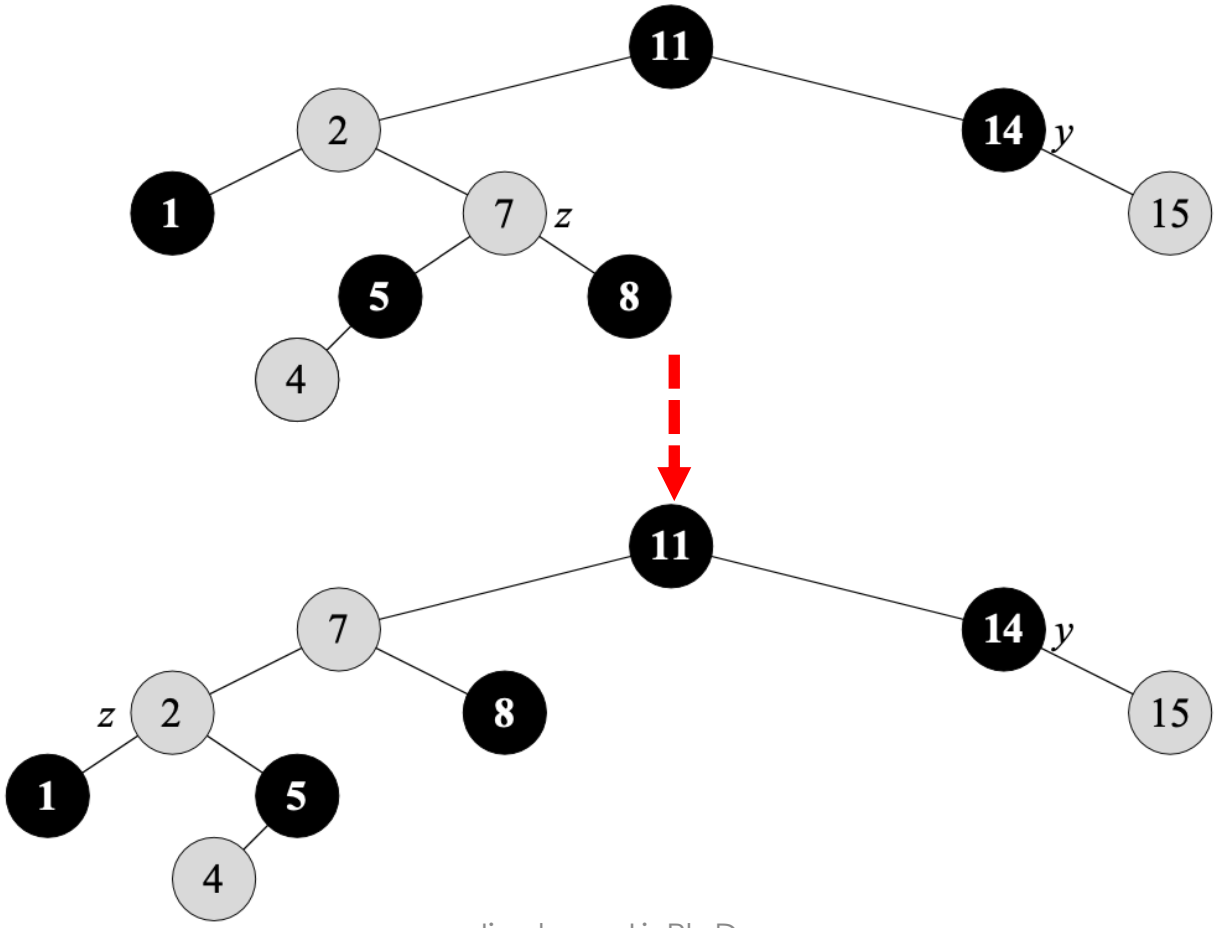
Insertion - Practice



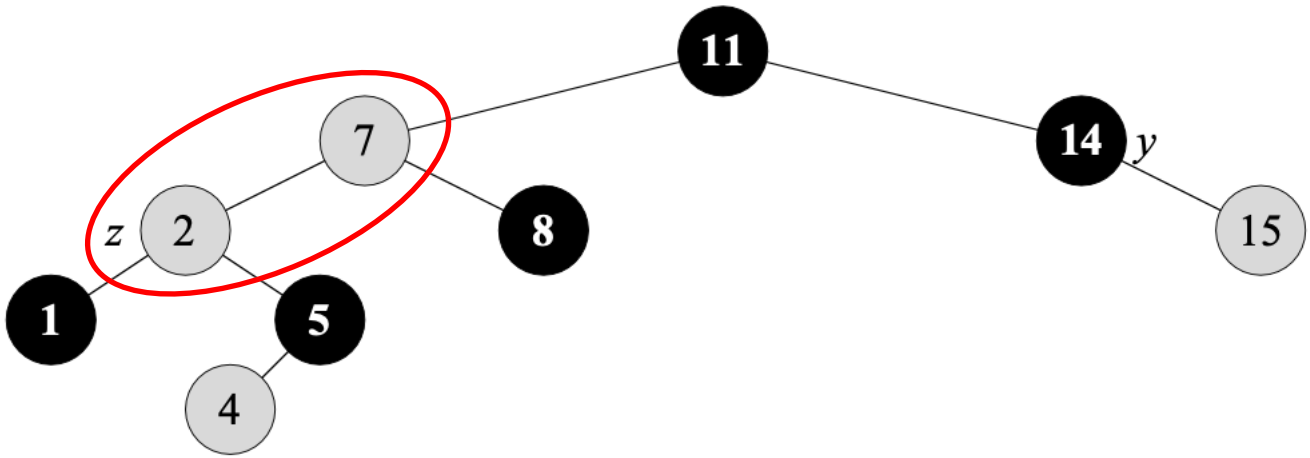
Insertion - Practice



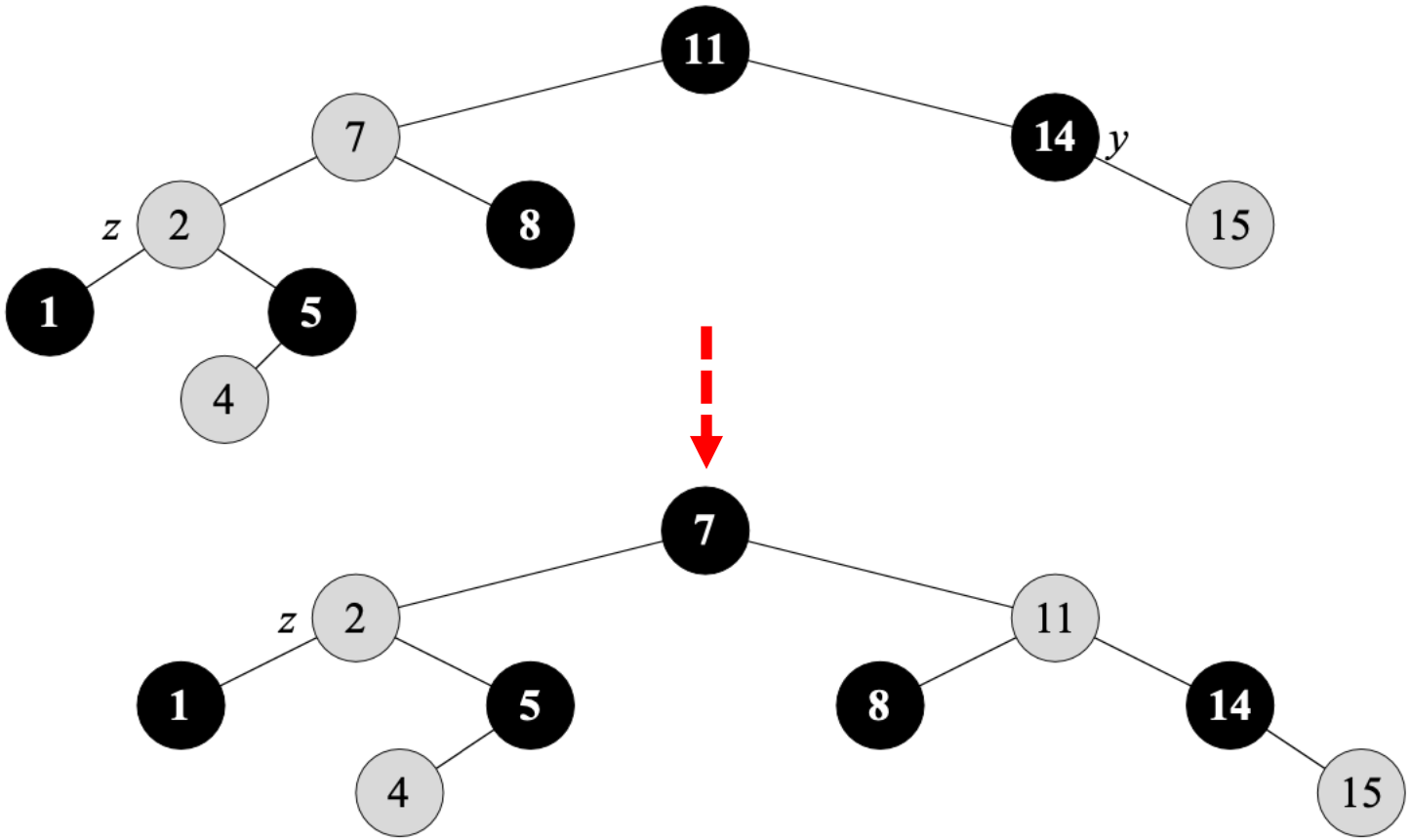
Insertion - Practice



Insertion - Practice



Insertion - Practice



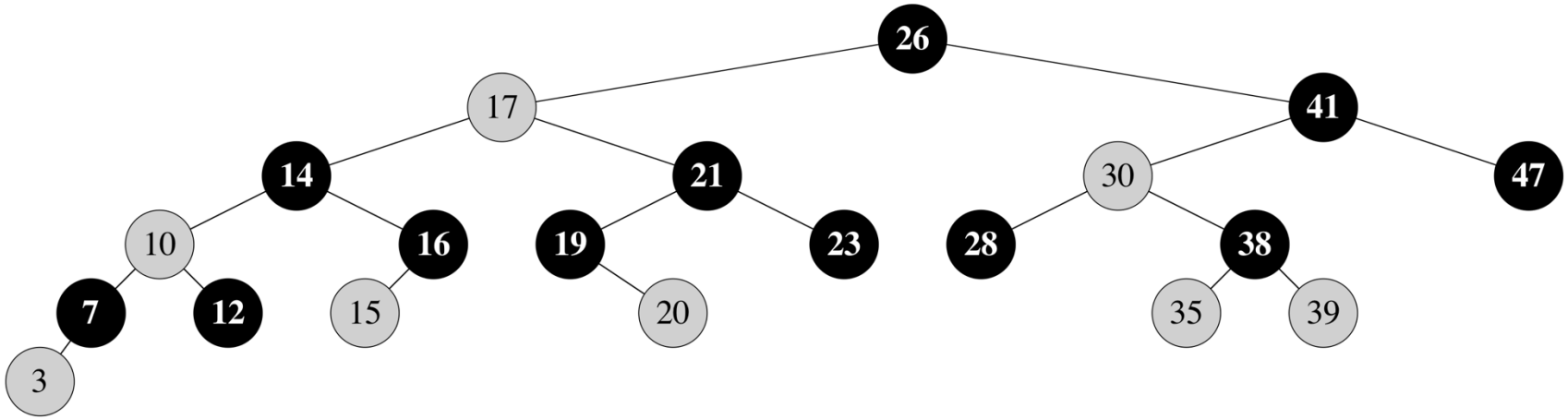
RBT - Practice

- Show the red-black tree that result after successively inserting the following keys into an initially empty red-black tree:

41, 38, 31, 12, 9, 8

RBT - Practice

Draw the red-black tree that results after TREE_INSERT is called on the tree below with key 36. If the inserted node is colored red, is the resulting tree a red-back tree? What if it is colored black?



Questions?

ROLL TIDE®